E1CSA313 DESIGN PROJECT-1 · ALLIANCE UNIVERSITY

The Blockchain in Chain of Record

Which real blockchain we use, how our application writes certificates into it, which blocks and data it checks, and the exact steps it takes to verify a certificate.

Ethereum	**1,604**	**17**	**86**
Sepolia public network	certificates recorded	Merkle batches, one per university	revocations, in 23 blocks

Smart contract	`CredentialRegistry` at `0x2E36EB891D77c7f9dE0BC1b00D17ECb00f37D15E`
Network	Ethereum Sepolia (chain id 11155111), Proof of Stake, a block about every 12 seconds
Deployed	Block 12,054,016, transaction `0x2ac18effe71f…3a003e5fe`
See it yourself	`sepolia.etherscan.io/address/0x2E36EB891D77c7f9dE0BC1b00D17ECb00f37D15E`
Website	`blockchain-ecru-three.vercel.app`: Blockchain page `/chain`, verifier `/verify`

Team: Manu M P · Tarun Kumar S · Anilkumar U K · Pruthviraj D N
Mentor: Dr. Saravanan P

All block numbers, hashes and counts in this document were read from Ethereum Sepolia on 3 November 2026, after the Karnataka Degree Registry dataset (synthetic) was imported.

1. The real blockchain we use

Chain of Record stores its certificates on **Ethereum**, the public blockchain, on its test network **Sepolia**. It is not a simulation and not a blockchain we run ourselves:

Property	Ethereum Sepolia
Who runs it	Thousands of independent nodes around the world. We run none of them; our application only connects to one through a public RPC address.
Consensus	Proof of Stake: validators propose and attest to blocks. A new block every 12 seconds.
Blocks	Each block has a number, its own hash, the **hash of its parent block**, a timestamp and the transactions in it. Our transactions share blocks with other people's: block 12,054,067, which holds Alliance University's batch, contains 151 transactions.
Signatures	Every transaction is signed with the sender's private key (ECDSA, secp256k1). The contract sees the signer's address as `msg.sender`.
Difference from mainnet	Only the currency: Sepolia ETH is free test money. The software, consensus and block structure are the same as Ethereum mainnet.

Our part of Ethereum: one smart contract

`backend/contracts/CredentialRegistry.sol` (Solidity 0.8, OpenZeppelin `Ownable2Step`, `Pausable`, `MerkleProof`). Once deployed, its code cannot be changed, and every node on the network runs it to check each transaction sent to it.

Function	Who may call it	What it does
`setIssuer(address, name, active)`	Contract owner (platform admin)	Authorise or suspend a university's wallet address
`issueCredential(id, docHash)`	An authorised university	Record one certificate's document hash
`anchorBatch(root, count)`	An authorised university	Record many certificates under one Merkle root
`revokeCredential` / `revokeBatchCredential`	Only the original issuer	Mark a certificate revoked, storing `keccak256(reason)`
`getCredential` / `verifyBatchCredential`	Anyone, free	Return the hash, issuer, whether the issuer is still active, issue and revoke times
`pause()` / `unpause()`	Owner	Emergency stop for issuing and revoking; verifying keeps working

The contract itself refuses bad writes: `NotAuthorizedIssuer`, `CredentialAlreadyExists`, `HashAlreadyRegistered`, `NotOriginalIssuer`, `AlreadyRevoked`, `RootAlreadyAnchored`, `InvalidProof`. A refused transaction changes nothing.

What is on the blockchain and what is not

Data	Where
Document hash (docHash) or Merkle root, issuer address, issued and revoked time, status, `keccak256` of the revocation reason	**Ethereum** (public, no personal data)

The marks card document, encrypted with AES-256-GCM	**IPFS** (through Pinata)
Names, marks, dates, reason text, Merkle proofs, users, audit log	**PostgreSQL** (Supabase)
Private keys of universities and the platform	Each user's own **MetaMask**. The server holds none.

2. How a certificate goes into the blockchain

Registrar
fills in the certificate in the Credential Center
→
API server
builds the document, computes its SHA-256 docHash
→
MetaMask
university signs `issueCredential` or `anchorBatch`
→
Ethereum
validators put it in a block; every node runs the contract
→
API server
reads the contract back, then saves the record and pins to IPFS

1. **Build the document and its hash.**
   The server joins the certificate's canonical line `cert_no|reg_no|NAME|dob|program|score|year|univ` and every semester row (semester, credits, SGPA, result) into one text.
   docHash = SHA-256 of that text.
   Changing a single SGPA gives a completely different docHash. (`POST /api/credentials/prepare`, `backend/server/hash.js`)

2. **The university signs in MetaMask.**
   The website asks MetaMask to call `issueCredential(id, docHash)` on the contract. MetaMask checks it is on Sepolia and on the account registered for that university, then signs with the university's private key. (`src/lib/wallet.js`)

3. **Ethereum mines it into a block.**
   The contract runs on every node: is the sender an authorised issuer, is the ID new, is the hash new, is the registry not paused? If any rule fails, the transaction is reverted and nothing is stored.

4. **The server confirms by reading the blockchain, not by trusting the browser.**
   It loads the transaction receipt and the contract: the transaction succeeded, it was sent to *our* contract, the contract now holds *this* ID with *this* docHash, signed by *this* university's address. Only then is the record saved. (`POST /api/credentials`, `backend/server/chain.js`)

5. **The document goes to IPFS, encrypted.**
   The server encrypts the document with a fresh AES-256-GCM key and pins it to IPFS. The key goes only into the student's QR link, after the `#`, which browsers never send to any server.

Batches: 1,604 certificates in 17 transactions

For many certificates at once, the university signs **one** `anchorBatch(root, count)`. Each certificate becomes a leaf `keccak256(keccak256(credentialId, docHash))`; leaves are hashed in pairs up to a single **Merkle root**, and only the root is stored on-chain. One certificate is later proved with its leaf and a handful of sibling hashes (7 for Alliance University's 117 certificates).

```
              Merkle root  (stored in the contract, batch 2)
              0x814445ad…1925686
             /                  \
       hash(A,B)                hash(C,D)          ← each parent = keccak256 of its two children
       /      \                 /      \
  leaf A    leaf B         leaf C    leaf D       ← leaf = keccak256(keccak256(id, docHash))
(cert 1)   (cert 2)       (cert 3)  (cert 4)

To prove cert 3: give leaf C + sibling D + sibling hash(A,B). The contract recomputes the root
and compares. Any changed certificate changes its leaf, so the root no longer matches.
```

Revoking

The issuing university signs `revokeCredential(id, keccak256(reason))` (or `revokeBatchCredential` with the Merkle proof). The contract refuses it from any other address (`NotOriginalIssuer`). Nothing is deleted: the issue and the revocation both stay on the chain as events, with their blocks.

3. The blocks our application wrote

Importing the Karnataka dataset: the platform owner authorised 17 university addresses with `setIssuer`, then each university signed one Merkle batch with its own key.

Batch	University	Certificates	Block	Transaction	Block hash
1	VTU	131	12,054,064	`0xb3090ffa…fa703a1d`	`0x5dc89798…9d53b44a`
2	Alliance University	117	12,054,067	`0x187e210c…c1edfb66`	`0x505a34d0…2aca75a4`
3	RV University	25	12,054,069	`0xbd8df92b…cc4a5150`	`0xe772e822…a8f82702`
4	IIT Dharwad	90	12,054,071	`0x80fb9aba…9a40a9dd`	`0x5eed875a…a829221f`
5	NIT Karnataka	156	12,054,073	`0x26edd1f4…45bc45d8`	`0x25a5fc31…0bfda698`
6	IIIT Bangalore	62	12,054,077	`0xae911e82…352cd668`	`0xabbabc31…2cf859cd`
7	IIIT Dharwad	24	12,054,080	`0x2bb08676…39296cb2`	`0xa4e1c804…7f913a09`
8	PES University	135	12,054,081	`0x7ac15895…3da521d5`	`0x27bc1fe4…5aa6a1d3`
9	IISc	105	12,054,084	`0x34440bfd…d05fe587`	`0xf05714c0…e1452db5`
10	Bangalore University	123	12,054,086	`0xb3a56889…f65b8a0f`	`0x7a3871b5…29de5259`
11	University of Mysore	89	12,054,088	`0x92849542…8939c8cd`	`0xdf2b13d0…787befb3`
12	Christ University	100	12,054,091	`0xe17239c5…43f4309f`	`0x4ff2a8c1…d818b6cc`
13	MAHE	121	12,054,094	`0x82133e7e…e7518768`	`0xf1fc812d…df15aa91`
14	REVA University	87	12,054,341	`0x3345bf91…35efef58`	`0xcaeaca67…e9f034ab`
15	Jain University	80	12,054,343	`0xa1c71ce5…64387484`	`0xb0f3cac7…d77d1b63`
16	Dayananda Sagar University	78	12,054,345	`0x57b2ed8e…448a55b5`	`0xd7e75d60…6c8f027e`
17	M S Ramaiah University	81	12,054,347	`0x9c8d0ef3…5cae6166`	`0xca77cc99…489a741c`
	Total	**1,604**	**then 86 revocations, each signed by its own university, in 23 blocks (from block 12,054,350)**		

How blocks are linked: a real example from our import

Batch 7 and batch 8 landed in consecutive blocks. Block 12,054,081 stores, as its **parent hash**, the exact hash of block 12,054,080:

Block 12,054,080	
contains	batch 7 (IIIT Dharwad)
parent	`0x6029e575…da1c`
block hash	`0xa4e1c804…3a09`

→

Block 12,054,081	
contains	batch 8 (PES University)
parent	`0xa4e1c804…3a09`
block hash	`0x27bc1fe4…a1d3`

Why an old certificate cannot be changed. Edit batch 7's root → the transactions in block 12,054,080 change → its block hash changes → block 12,054,081's parent hash no longer matches → every later block (over 300 so far) would have to be rebuilt and accepted by Sepolia's validators. Nobody, including us, can do that.

On the website's Blockchain page, click any block number in "Latest contract events" to see its hash and parent hash, read live from Ethereum.

4. What the application checks on the blockchain

Every verification ends in one of three answers: **VALID** **REVOKED** **INVALID**. These are the blockchain checks behind them:

#	Check	Read from Ethereum	If it fails
1	The certificate is recorded in the contract (individually, or its Merkle proof leads to an anchored batch root)	`getCredential` / `verifyBatchCredential`	**INVALID** not on chain
2	The recorded hash is exactly the docHash of this certificate's details and marks	Contract storage: `docHash` or the Merkle root	**INVALID** hash mismatch
3	It was signed by an address the platform authorised, and that university is still active	`issuers(address)`	**INVALID** untrusted issuer
4	The transaction that recorded it succeeded, was sent to our contract, and is in a real block	Transaction receipt → **block number**, **block hash**	record not accepted
5	How deep that block is: confirmations = latest block − its block + 1	`eth_blockNumber`	shown in the proof
6	Whether the issuing university has revoked it	`status` / `revokedAt`	**REVOKED**

If Ethereum does not answer, the API returns **503 unavailable**, never a verdict. A missing answer is never treated as "valid".

A real check: certificate AU/DC/2016/000759 (Alliance University)

docHash	`0x0d9ef85e4fa7ab55588e47c46c89dcb7ea81b5040ea5e603cf81148dd3157c76`
In batch	2, position 56 of 117; Merkle proof of 7 sibling hashes → root `0x814445ad…1925686`; **proof valid**
Transaction	`0x187e210c4ce75d3304bf69747e2737b459a73e8a2c25382e6eba2942c1edfb66`
Block	12,054,067 · hash `0x505a34d061ea…2aca75a4` · 3 Nov 2026 08:21:36 UTC

Confirmations	318 at the time of this check (it grows by one every 12 seconds)
Issuer	`0x987184DeCCf51cEE4fc60df25115f318b7b637ec` · "Alliance University" · active
Revoked	No (`revokedAt = 0`)
Answer	**VALID**

A real revoked certificate: AU/DC/2016/000060

Revoked by	The same Alliance University address (the contract allows no one else)
Revocation transaction	`0x8dac54a814b3c7a9209e5d08ef0478b09096335da24d0201d27a0f8a8c082d68`
Block	12,054,350
Reason	"Malpractice confirmed post-result". Only `keccak256` of this text is on-chain; the text is in the database and must hash to the same value.
Answer	**REVOKED** : genuine certificate, withdrawn by its university

5. Step by step: how a certificate is checked

The verifier portal (`/verify`) offers three ways in. All three end at the smart contract.

A. Certificate photo (main flow)

1. **Read the photo in the browser.**
   Tesseract OCR reads the text in three passes, and the QR code is decoded. The photo itself is never uploaded; only the text read from it is sent. (`src/lib/photoEvidence.js`)
   No certificate number found → INVALID "unreadable"
2. **Find the certificate number in the smart contract.**
   The number becomes the credential ID; the server reads the contract (individual record, or the batch with the Merkle proof).
   Not in the contract → INVALID "not on chain"
3. **Check the issuer.**
   The address that signed it must be authorised and active in the contract.
   → INVALID "untrusted issuer"
4. **Compare every printed detail**
   (name, register number, programme, CGPA, year, date of birth, every semester's SGPA) with the university's record. Numbers must match exactly; names allow small OCR slips.
   A row that cannot be read → INVALID "could not read" · a detail that differs → INVALID "mismatch"
5. **Recompute the docHash**
   from the record and all semester rows, and compare it with the hash in the contract.
   Different → INVALID "hash mismatch" (someone edited the database)
6. **Compare the QR code's hash**
   with the same on-chain docHash.
   Different → INVALID "QR hash mismatch" (forged QR)
7. **Read the revocation status**
   from the contract.
   Revoked → REVOKED, with the reason

8. **Answer VALID**
   and show the proof: network, contract, transaction, block number, block hash, confirmations, batch, Merkle root and whether the proof is valid. (`backend/server/photoVerify.js`)

B. QR scan, without our server

The student's QR link is `…/verify/<cert no>/<docHash>#cid=<IPFS address>&k=<key>`. Everything runs in the verifier's browser (`src/lib/chainVerify.js`):

1. **Fetch the document.**
   Download the encrypted marks card from IPFS through any public gateway.
2. **Decrypt it.**
   Use the key from the QR link (AES-256-GCM, Web Crypto).
3. **Recompute the hash.**
   Its SHA-256 must equal the hash in the QR. Otherwise INVALID
4. **Ask Ethereum directly.**
   Through a public RPC: is this hash recorded (Merkle proof against the batch root), by an authorised issuer, and not revoked?
5. **Read the transaction's block.**
   Show its block number and confirmations, then VALID or REVOKED.

Our API server and database are not contacted at all, so this check works even if they are down.

C. Credential ID, certificate number or hash

1. Look the credential up in the contract (by ID or by docHash). Not found → INVALID
2. Check the issuer is authorised and active. → INVALID "untrusted issuer"
3. If the university's record exists, its hash must equal the contract's. → INVALID "record mismatch"
4. Return VALID or REVOKED from the contract's status, with the same proof as above.

6. What each attack runs into

Someone tries to…	…and the blockchain check that stops it
Edit a mark on a printed certificate	The printed detail no longer matches the record, and its docHash would not be in the contract → INVALID
Edit the record in our database	The record no longer hashes to the docHash stored on Ethereum → INVALID "hash mismatch"
Print a fake certificate with a made-up number	The number is not in the contract → INVALID "not on chain"
Paste a QR code from another certificate	The QR hash differs from the on-chain hash for this number → INVALID
Set up a fake "university" and issue	The contract rejects any address not authorised with `setIssuer` (`NotAuthorizedIssuer`)
Revoke another university's certificate	The contract rejects it (`NotOriginalIssuer`)

Hack our server to issue certificates	The server holds no private key; every write needs the university's MetaMask signature
Rewrite an old block	Every later block's parent hash breaks, and Sepolia's validators would have to accept the rewrite
Hide a revocation	The revocation is a permanent event in a mined block; the contract's status says revoked

Measured results

Test	Result
Automated tests: contract, API on Hardhat's local Ethereum, photo, IPFS, security	92 passing (99 with the database tests)
666 labelled verification requests against the contract	666 / 666 correct
Simulated certificate photos (OCR + QR), 42 per quality level	clean 42/42 · phone-like 42/42 · small and blurred 39/42

Limits, stated plainly

- **Sepolia is Ethereum's test network.** Real blocks and consensus; the ETH has no money value.
- **The API server and database are still central** for personal data, sign-in and photo checks. The contract, IPFS and the QR check do not depend on them.
- **It proves the document, not the person.** An exact copy of a genuine certificate is VALID.
- **Photos were simulated** by software, not taken with a camera; only this system's card layout is read.
- **The student wallet is an installable web app (PWA)**, not Flutter. Hyperledger, listed in the brief as an option, is not used.
- **The dataset is synthetic.** Certificates say "Not an official university document".

Chain of Record · E1CSA313 Design Project-1 · Alliance University · Source: `backend/contracts/CredentialRegistry.sol`, `backend/server/`, `src/lib/`